MemGPT: Towards LLMs as Operating Systems

Charles Packer [1] **Sarah Wooders** [1] **Kevin Lin** [1]
Vivian Fang [1] **Shishir G. Patil** [1] **Ion Stoica** [1] **Joseph E. Gonzalez** [1]

Abstract

Large language models (LLMs) have revolutionized AI, but are constrained by limited context windows, hindering their utility in tasks like extended conversations and document analysis. To enable using context beyond limited context windows, we propose *virtual context management*, a technique drawing inspiration from hierarchical memory systems in traditional operating systems which provide the illusion of an extended virtual memory via paging between physical memory and disk. Using this technique, we introduce MemGPT (MemoryGPT), a system that intelligently manages different storage tiers in order to effectively provide extended context within the LLM's limited context window. We evaluate our OS-inspired design in two domains where the limited context windows of modern LLMs severely handicaps their performance: document analysis, where MemGPT is able to analyze large documents that far exceed the underlying LLM's context window, and multi-session chat, where MemGPT can create conversational agents that remember, reflect, and evolve dynamically through long-term interactions with their users. We release MemGPT code and data for our experiments at https://research.memgpt.ai.

1. Introduction

In recent years, large language models (LLMs) and their underlying transformer architecture (Vaswani et al., 2017; Devlin et al., 2018; Brown et al., 2020; Ouyang et al., 2022) have become the cornerstone of conversational AI and have led to a wide array of consumer and enterprise applications. Despite these advances, the limited fixed-length context windows used by LLMs significantly hinders their applicability to long conversations or reasoning about long documents. For example, the most widely used open-source LLMs can only support a few dozen back-and-forth messages or reason about a short document before exceeding their maximum input length (Touvron et al., 2023).

Directly extending the context length of transformers incurs a quadratic increase in computational time and memory cost due to the transformer architecture's self-attention mechanism, making the design of new long-context architectures a pressing research challenge (Dai et al., 2019; Kitaev et al., 2020; Beltagy et al., 2020). While developing longer models is an active area of research (Dong et al., 2023), even if we could overcome the computational challenges of context scaling, recent research shows that long-context models struggle to utilize additional context effectively (Liu et al., 2023a). As consequence, given the considerable resources needed to train state-of-the-art LLMs and diminishing returns of context scaling, there is a critical need for alternative techniques to support long context.

In this paper, we study how to provide the illusion of an infinite context while continuing to use fixed-context models. Our approach borrows from the idea of virtual memory paging that was developed to enable applications to work on datasets that far exceed the available memory by paging data between main memory and disk. We leverage the recent progress in function calling abilities of LLM agents (Schick et al., 2023; Liu et al., 2023b) to design MemGPT, an OS-inspired LLM system for **virtual context management**. Using function calls, LLM agents can read and write to external data sources, modify their own context, and choose when to return responses to the user.

These capabilities allow LLMs to effective "page" in and out information between context windows (analogous to "main memory" in operating systems) and external storage, similar to hierarchical memory in traditional OSes. In addition, function calls can be leveraged to manage control flow between context management, response generation, and user interactions. This allows for an agent to choose to *iteratively* modify what is in its context for a single task, thereby more effectively utilizing its limited context.

In MemGPT, we treat context windows as a constrained memory resource, and design a memory hiearchy for LLMs analogous to memory tiers used in traditional OSes (Patterson et al., 1988). Applications in traditional OSes interact

[1]University of California, Berkeley. Correspondence to: Charles Packer <cpacker@berkeley.edu>.

arXiv:2310.08560v2 [cs.AI] 12 Feb 2024

MemGPT：迈向大语言模型作为操作系统

查尔斯·帕克 [1] **莎拉·伍德尔斯** [1] **凯文·林** [1] **菲比安·方** [1] **希希尔·G·帕蒂尔** [1] **伊昂·斯托伊卡** [1] **约瑟夫·E·冈萨雷斯** [1]

摘要

大语言模型（LLMs）革新了人工智能，但其受限于有限的上下文窗口，这阻碍了它们在扩展对话和文档分析等任务中的应用。为了使用超出有限上下文窗口的上下文，我们提出了<样式 id='1'>虚拟上下文管理</样式>，这是一种从传统操作系统中分层内存系统汲取灵感的技巧，该系统通过物理内存和磁盘之间的分页提供扩展虚拟内存的错觉。使用这种技巧，我们引入了MemGPT（MemoryGPT），这是一个智能管理不同存储层以在LLM有限的上下文窗口内有效提供扩展上下文的系统。我们在两个领域评估了我们的受操作系统启发的设计，在这些领域，现代LLM有限的上下文窗口严重限制了它们的性能：文档分析，其中MemGPT能够分析远超底层LLM上下文窗口的大型文档，以及多会话聊天，其中MemGPT能够创建对话代理，这些代理能够通过与其用户的长期互动记住、反思和动态演变。我们在https://research.memgpt.ai上发布了我们的实验MemGPT代码和数据。

1. 引言

近年来，大语言模型（LLMs）及其底层Transformer架构（Vaswani等人，2017年；Devlin等人，2018年；Brown等人，2020年；Ouyang等人，2022年）已成为对话式AI的基石，并推动了各种消费者和企业应用。尽管取得了这些进展，但LLMs使用的固定长度上下文窗口有限，这严重阻碍了它们在长对话或长文档推理方面的适用性。例如，最广泛使用的开源大语言模型只能支持几十个来回的消息，或在超出其最大输入长度之前对短文档进行推理（Touvron等人，2023年）。

直接扩展转换器的上下文长度，由于转换器架构的自注意力机制，会导致计算时间和内存成本呈平方级增长，这使得设计新的长上下文架构成为一个紧迫的研究挑战（Dai等人，2019；Kitaev等人，2020；Beltagy等人，2020）。虽然开发更长的模型是一个活跃的研究领域（Dong等人，2023），即使我们能克服上下文扩展的计算挑战，最近的研究表明，长上下文模型难以有效利用额外的上下文（Liu等人，2023a）。作为结果，考虑到训练最先进的大语言模型所需的巨大资源以及上下文扩展的边际效益递减，迫切需要支持长上下文的替代技术。

在本文中，我们研究了如何在继续使用固定上下文模型的同时提供无限上下文的错觉。我们的方法借鉴了虚拟内存分页的思想，该思想是为了使应用程序能够在远超可用内存的数据集上工作而开发的，通过在主内存和磁盘之间分页数据来实现。我们利用大语言模型代理在函数调用能力方面的最新进展（Schick等人，2023；Liu等人，2023b）来设计MemGPT，这是一个受操作系统启发的用于<code style id='1'>虚拟上下文管理</code>的大语言模型系统。通过函数调用，大语言模型代理可以读取和写入外部数据源，修改自己的上下文，并选择何时向用户返回响应。

这些功能使大语言模型能够有效地在上下文窗口（类似于操作系统中的“主内存”）和外部存储之间“分页”信息，类似于传统操作系统中的分层内存。此外，函数调用可以用来在上下文管理、响应生成和用户交互之间管理控制流。这允许代理选择为单个任务<code style id='1'>迭代</code>地修改其上下文中的内容，从而更有效地利用其有限的上下文。

在MemGPT中，我们将上下文窗口视为一种受限的内存资源，并设计了一种类似传统OS中内存层级（Patter- son等人，1988年）的内存层次结构用于大语言模型。传统OS中的应用程序与

[1]加州大学伯克利分校。通讯联系：CharlesPacker <cpacker@berkeley.edu>。

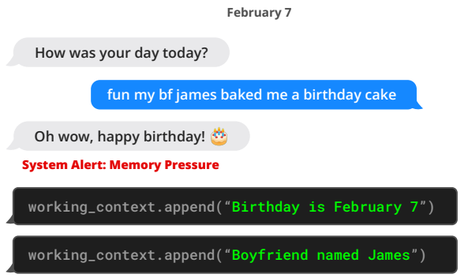

Figure 1. MemGPT (left) writes data to persistent memory after it receives a system alert about limited context space.

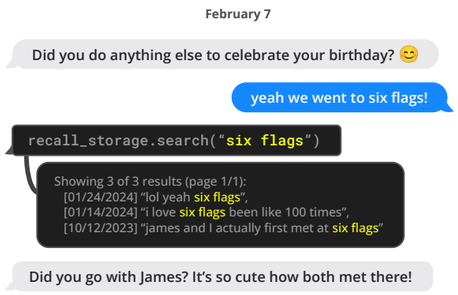

Figure 2. MemGPT (left) can search out-of-context data to bring relevant information into the current context window.

with *virtual memory*, which provides an illusion of there being more memory resources than are actually available in physical (i.e., main) memory by the OS paging overflow data to disk and retrieving data (via a page fault) back into memory when accessed by applications. To provide a similar illusion of longer context length (analogous to virtual memory), we allow the LLM to manage what is placed in its own context (analogous to physical memory) via an 'LLM OS', which we call MemGPT. MemGPT enables the LLM to retrieve relevant historical data missing from what is placed in-context, and also evict less relevant data from context and into external storage systems. Figure 3 illustrates the components of MemGPT.

The combined use of a memory-hierarchy, OS functions and event-based control flow allow MemGPT to handle unbounded context using LLMs that have finite context windows. To demonstrate the utility of our new OS-inspired LLM system, we evaluate MemGPT on two domains where the performance of existing LLMs is severely limited by finite context: document analysis, where the length of standard text files can quickly exceed the input capacity of modern LLMs, and conversational agents, where LLMs bound by limited conversation windows lack context awareness, persona consistency, and long-term memory during extended conversations. In both settings, MemGPT is able to overcome the limitations of finite context to outperform existing LLM-based approaches.

2. MemGPT (MemoryGPT)

MemGPT's OS-inspired multi-level memory architecture delineates between two primary memory types: **main context** (analogous to main memory/physical memory/RAM) and **external context** (analogous to disk memory/disk storage). Main context consists of the LLM *prompt tokens*—anything in main context is considered *in-context* and can be accessed by the LLM processor during inference. External context refers to any information that is held outside of the LLMs fixed context window. This *out-of-context* data must always be explicitly moved into main context in order for it to be passed to the LLM processor during inference. MemGPT provides function calls that the LLM processor to manage its own memory without any user intervention.

2.1. Main context (*prompt tokens*)

The prompt tokens in MemGPT are split into three contiguous sections: the **system instructions**, **working context**, and **FIFO Queue**. The system instructions are read-only (static) and contain information on the MemGPT control flow, the intended usage of the different memory levels, and instructions on how to use the MemGPT functions (e.g. how to retrieve out-of-context data). Working context is a fixed-size read/write block of unstructured text, writeable only via MemGPT function calls. In conversational settings, working context is intended to be used to store key facts, preferences, and other important information about the user and the persona the agent is adopting, allowing the agent to converse fluently with the user. The FIFO queue stores a rolling history of messages, including messages between the agent and user, as well as system messages (e.g. memory warnings) and function call inputs and outputs. The first index in the FIFO queue stores a system message containing a recursive summary of messages that have been evicted from the queue.

2.2. Queue Manager

The queue manager manages messages in **recall storage** and the **FIFO queue**. When a new message is received by the system, the queue manager appends the incoming messages to the FIFO queue, concatenates the prompt tokens and triggers the LLM inference to generate LLM output (the completion tokens). The queue manager writes both the incoming message and the generated LLM output to recall storage (the MemGPT message database). When messages in recall storage are retrieved via a MemGPT function call, the queue manager appends them to the back of

图*1*。 MemGPT（左图）在接收到关于上下文空间有限的系统警报后，将数据写入持久化内存。

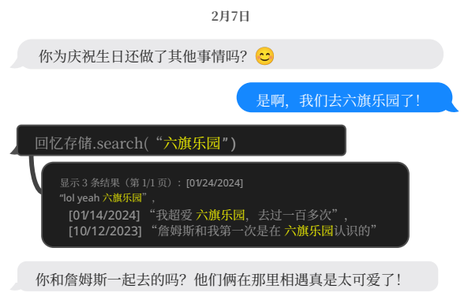

图2。 MemGPT（左图）可以搜索上下文外数据，将相关信息带入当前上下文窗口。

虚拟内存交互，该机制通过OS将页面溢出数据交换到磁盘，并在应用程序访问时通过页面错误将数据重新从内存中检索回来，从而营造出比实际物理内存（即主内存）更多的内存资源假象。为了提供类似的更长的上下文长度假象（类似于虚拟内存），我们允许大语言模型通过一个‘大语言模型OS’来管理其上下文中放置的内容（类似于物理内存），我们称之为MemGPT。MemGPT使大语言模型能够检索上下文中缺失的相关历史数据，同时将不太相关的数据从上下文中移除并存储到外部存储系统中。图3展示了MemGPT的组件。

内存层次结构、操作系统函数和基于事件的控制流程的结合使用，使MemGPT能够利用具有有限上下文窗口的大语言模型来处理无界上下文。为了展示我们新的受操作系统启发的LLM系统的实用性，我们在两个领域评估了MemGPT，现有LLM的性能在这两个领域中因有限上下文而严重受限：文档分析，其中标准文本文件的长度很快就会超过现代LLM的输入容量，以及对话代理，其中受限于有限对话窗口的LLM缺乏上下文感知、角色一致性和长期记忆，在长时间的对话中。在这两种设置下，MemGPT都能够克服有限上下文的限制，优于现有的基于LLM的方法。

2. MemGPT（MemoryGPT）

MemGPT 的受操作系统启发的多级内存架构区分了两种主要内存类型：**主上下文**（类似于主内存/物理内存/RAM）和**外部上下文**（类似于磁盘内存 / 磁盘存储）。主上下文由大语言模型的 提示令牌 组成——主上下文中的任何内容都被视为 上下文内，并且在推理过程中可以被大语言模型处理器访问。外部上下文指的是任何存储在大语言模型固定上下文窗口之外的信息。这种 上下文外 数据 必须始终显式地移入主上下文，以便在推理过程中传递给大语言模型处理器。MemGPT 提供了函数调用，使大语言模型处理器能够自行管理内存，无需任何用户干预。

2.1. 主上下文（提示令牌）

MemGPT中的提示令牌被分为三个连续的部分：系统指令、**工作上下文**和**先进先出队列**。系统指令是只读的（静态的），包含有关MemGPT控制流、不同内存级别的预期用途以及如何使用MemGPT函数（例如如何检索上下文外数据）的信息。工作上下文是一个固定大小的可读可写非结构化文本块，只能通过MemGPT函数调用进行写入。在对话场景中，工作上下文旨在存储关键事实、偏好以及有关用户和代理所采用角色的重要信息，使代理能够与用户流畅对话。先进先出队列存储消息的滚动历史记录，包括代理和用户之间的消息，以及系统消息（例如内存警告）和函数调用输入输出。FIFO队列中的第一个索引存储一条系统消息，其中包含从队列中移除的消息的递归摘要。

2.2. 队列管理器

队列管理器管理 **召回存储** 中的消息以及 **先进先出队列**。当系统接收到新消息时，队列管理器将收到的消息追加到先进先出队列中，连接提示令牌并触发大语言模型推理，生成大语言模型输出（完成令牌）。队列管理器将收到的消息和生成的大语言模型输出都写入召回存储（MemGPT消息数据库）。当通过MemGPT函数调用检索召回存储中的消息时，队列管理器将它们追加到队列末尾，以便重新插入大语言模型的上下文窗口。

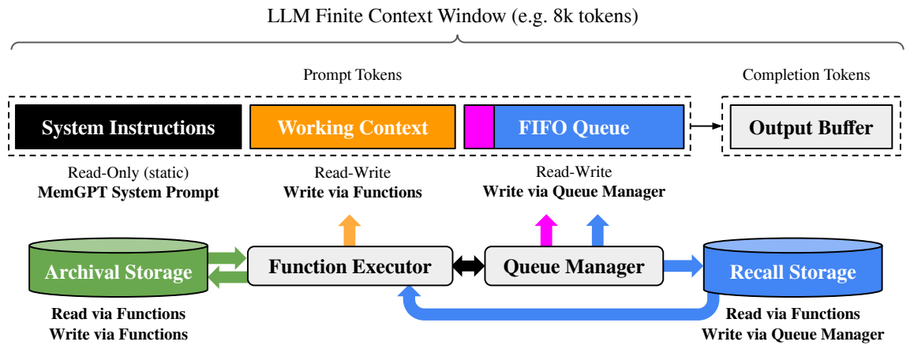

Figure 3. In MemGPT, a fixed-context LLM processor is augmented with a hierarchical memory system and functions that let it manage its own memory. The LLM's prompt tokens (inputs), or *main context*, consist of the system instructions, working context, and a FIFO queue. The LLM completion tokens (outputs) are interpreted as function calls by the function executor. MemGPT uses functions to move data between main context and *external context* (the archival and recall storage databases). The LLM can request immediate follow-up LLM inference to chain function calls together by generating a special keyword argument (`request_heartbeat=true`) in its output; function chaining is what allows MemGPT to perform multi-step retrieval to answer user queries.

the queue to reinsert them into the LLM's context window.

The queue manager is also responsible for controlling context overflow via a queue eviction policy. When the prompt tokens exceed the 'warning token count' of the underlying LLM's context window (e.g. 70% of the context window), the queue manager inserts a system message into the queue warning the LLM of an impending queue eviction (a 'memory pressure' warning) to allow the LLM to use MemGPT functions to store important information contained in the FIFO queue to working context or **archival storage** (a read/write database storing arbitrary length text objects). When the prompt tokens exceed the 'flush token count' (e.g. 100% of the context window), the queue manager flushes the queue to free up space in the context window: the queue manager evicts a specific count of messages (e.g. 50% of the context window), generates a new recursive summary using the existing recursive summary and evicted messages. Once the queue is flushed, the evicted messages are no longer in-context and immediately viewable to the LLM, however they are stored indefinitely in recall storage and readable via MemGPT function calls.

2.3. Function executor (handling of *completion tokens*)

MemGPT orchestrates data movement between main context and external context via function calls that are generated by the LLM processor. Memory edits and retrieval are entirely self-directed: MemGPT autonomously updates and searches through its own memory based on the current context. For instance, it can decide when to move items between contexts (e.g. when the conversation history is becoming too long, as show in Figure 1) and modify its main context to better reflect its evolving understanding of its current objectives and responsibilities (as shown in Figure 3). We implement self-directed editing and retrieval by providing explicit instructions within the system instructions that guide the LLM on how to interact with the MemGPT memory systems. These instructions comprise two main components: (1) a detailed description of the memory hierarchy and their respective utilities, and (2) a function schema (complete with their natural language descriptions) that the system can call to access or modify its memory.

During each inference cycle, LLM processor takes main context (concatenated into a single string) as input, and generates an output string. This output string is parsed by MemGPT to ensure correctness, and if the parser validates the function arguments the function is executed. The results, including any runtime errors that occur (e.g. trying to add to main context when it is already at maximum capacity), are then fed back to the processor by MemGPT. This feedback loop enables the system to learn from its actions and adjust its behavior accordingly. Awareness of context limits is a key aspect in making the self-editing mechanism work effectively, to this end MemGPT prompts the processor with warnings regarding token limitations to guide its memory management decisions. Additionally, our memory retrieval mechanisms are designed to be cognizant of these token constraints and implement pagination to prevent retrieval calls from overflowing the context window.

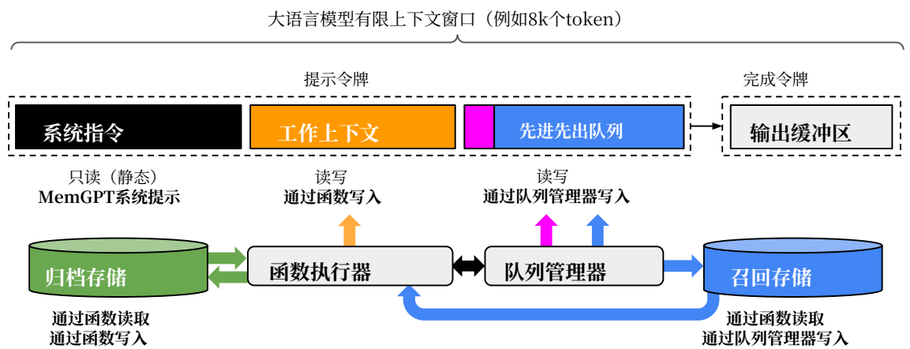

图*3*。 在MemGPT中，一个固定上下文的大语言模型处理器被一个分层内存系统和让它能管理自己内存的函数所增强。大语言模型的提示令牌（输入），或 主上下文，由系统指令、工作上下文和一个先进先出队列组成。大语言模型完成令牌（输出）被函数执行器解释为函数调用。MemGPT使用函数在主上下文和 外部上下文 (归档和召回存储数据库) 之间移动数据。大语言模型可以请求立即的后续大语言模型推理来将函数调用链接在一起，通过在其输出中生成一个特殊的关键字参数（请求心跳=true）；函数链接是MemGPT能够执行多步检索来回答用户查询的原因。

将其重新插入大语言模型的上下文窗口。

队列管理器还负责通过队列驱逐策略控制上下文溢出。当提示令牌超过底层大语言模型上下文窗口的“警告令牌计数”（例如上下文窗口的70%），队列管理器会向队列中插入一条系统消息，警告大语言模型即将发生队列驱逐（一个“内存压力”警告），以允许大语言模型使用MemGPT函数将FIFO队列中包含的重要信息存储到工作上下文或**归档存储**（一个可读写的存储任意长度文本对象的数据库）。当提示令牌超过“刷新令牌计数”（例如上下文窗口的100%），队列管理器会刷新队列以释放上下文窗口中的空间：队列管理器驱逐特定数量的消息（例如上下文窗口的50%），使用现有的递归摘要和被驱逐的消息生成一个新的递归摘要。一旦队列被刷新，被驱逐的消息就不再在上下文中，并且可以立即被大语言模型查看，但是它们会无限期地存储在召回存储中，并且可以通过MemGPT函数调用读取。

2.3. 函数执行器（处理<样式 id='1'>完成令牌</样式>）

MemGPT 通过大语言模型处理器生成的函数调用，在主上下文和外部上下文之间协调数据移动。内存编辑和检索完全是自我驱动的：MemGPT 根据当前上下文自主更新和搜索自己的内存。例如，它可以决定何时在上下文之间移动项目（例如，当对话历史变得过长时，如图1所示），并修改其主上下文以更好地反映其对当前目标和职责不断发展的理解（如图3所示）。我们通过在系统指令中提供明确的指令来实现自我驱动的编辑和检索，这些指令指导大语言模型如何与MemGPT内存系统交互。这些指令包含两个主要组成部分：(1) 内存层次结构的详细描述及其各自用途，以及 (2) 系统可以调用来访问或修改其内存的函数模式（包括其自然语言描述）。

我们通过在系统指令中提供明确的指令来实现自我驱动的编辑和检索，这些指令指导大语言模型如何与MemGPT内存系统交互。这些指令包含两个主要组成部分：(1) 内存层次结构的详细描述及其各自用途，以及 (2) 系统可以调用来访问或修改其内存的函数模式（包括其自然语言描述）。

在每次推理周期中，大语言模型处理器将主上下文（合并为单个字符串）作为输入，并生成一个输出字符串。该输出字符串由MemGPT解析以确保正确性，如果解析器验证了函数参数，则执行该函数。结果（包括任何运行时错误，例如当主上下文已达到最大容量时尝试添加内容）随后由MemGPT反馈给处理器。这种反馈循环使系统能够从其行为中学习并相应地调整其行为。了解上下文限制是使自我编辑机制有效工作的关键方面，为此MemGPT会向处理器发出关于令牌限制的警告，以指导其内存管理决策。此外，我们的内存检索机制被设计为意识到这些令牌约束，并实现分页以防止检索调用溢出上下文窗口。

Table 1. Comparing context lengths of commonly used models and LLM APIs (data collected 1/2024). *Approximate message count assuming a preprompt of 1k tokens, and an average message size of ~50 tokens (~250 characters). 'Open' means the model is open-source or open-weights (vs only available behind an API).

Model / API name	Open?	Context Window Tokens	*Messages
Llama (1)	✓	2k	20
Llama 2	✓	4k	60
GPT-3.5 Turbo (release)	✗	4k	60
Mistral 7B	✓	8k	140
GPT-4 (release)	✗	8k	140
GPT-3.5 Turbo	✗	16k	300
GPT-4	✗	32k	~600
Claude 2	✗	100k	~2000
GPT-4 Turbo	✗	128k	~2600
Yi-34B-200k	✓	200k	~4000

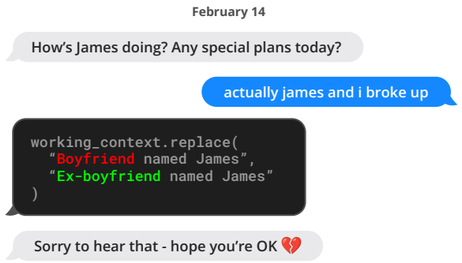

Figure 4. An example conversation snippet where MemGPT (left) updates stored information. Here the information is stored in working context memory (located within the prompt tokens).

2.4. Control flow and function chaining

In MemGPT, *events* trigger LLM inference: events are generalized inputs to MemGPT and can consist of user messages (in chat applications), system messages (e.g. main context capacity warnings), user interactions (e.g. an alert that a user just logged in, or an alert that they finished uploading a document), and timed events that are run on a regular schedule (allowing MemGPT to run 'unprompted' without user intervention). MemGPT processes events with a parser to convert them into plain text messages that can be appended to main context and eventually be fed as input into the LLM processor.

Many practical tasks require calling multiple functions in sequence, for example, navigating through multiple pages of results from a single query or collating data from different documents in main context from separate queries. Function chaining allows MemGPT to execute multiple function calls sequentially before returning control to the user. In MemGPT, functions can be called with a special flag that requests control be immediately returned to the processor after the requested function completes execution. If this flag is present, MemGPT will add the function output to main context and (as opposed to pausing processor execution). If this flag is not present (a *yield*), MemGPT will not run the LLM processor until the next external event trigger (e.g. a user message or scheduled interrupt).

3. Experiments

We assess MemGPT in two long-context domains: conversational agents and document analysis. For conversational agents, we expand the existing Multi-Session Chat dataset (Xu et al., 2021) and introduce two new dialogue tasks that evaluate an agent's ability to retain knowledge across long conversations. For document analysis, we benchmark MemGPT on existing tasks from (Liu et al., 2023a) for question answering and key-value retrieval over lengthy documents. We also propose a new nested key-value retrieval task requiring collating information across multiple data sources, which tests the ability of an agent to collate information from multiple data sources (multi-hop retrieval). We publicly release our augmented MSC dataset, nested KV retrieval dataset, and a dataset of embeddings for 20M Wikipedia articles to facilitate future research. Our code for the benchmarks is available at https://research.memgpt.ai.

Implementation details. When discussing OpenAI models, unless otherwise specified 'GPT-4 Turbo' refers to the specific `gpt-4-1106-preview` model endpoint (context window of 128,000), 'GPT-4' refers to `gpt-4-0613` (context window of 8,192), and 'GPT-3.5 Turbo' refers to `gpt-3.5-turbo-1106` (context window of 16,385). In experiments, we run MemGPT with all baseline models (GPT-4, GPT-4 Turbo, and GPT 3.5) to show how the underlying model performance affects MemGPT's.

3.1. MemGPT for conversational agents

Conversational agents like virtual companions and personalized assistants aim to engage users in natural, long-term interactions, potentially spanning weeks, months, or even years. This creates challenges for models with fixed-length contexts, which can only reference a limited history of the conversation. An 'infinite context' agent should seamlessly handle continuous exchanges without boundary or reset. When conversing with a user, such an agent must satisfy two key criteria: (1) Consistency - The agent should maintain conversational coherence. New facts, preferences, and events mentioned should align with prior statements from both the user and agent. (2) Engagement - The agent should draw on long-term knowledge about the user to personalize

表1。 比较常用模型和大语言模型API的上下文长度（数据收集于2024年1月）。*假设预提示为1k个token，平均消息大小为 ~50 个token（~250 个字符）。'开放'表示模型是开源或开放权重（与仅通过API提供相对）。

模型/API名称已打开?	开?	上下文窗口 令牌	*消息
Llama (1)	✓	2k	20
Llama 2	✓	4k	60
GPT-3.5 Turbo (发布)	✗	4k	60
Mistral 7B	✓	8k	140
GPT-4 (发布)	✗	8k	140
GPT-3.5 Turbo	✗	16k	300
GPT-4	✗	32k	~600
Claude 2	✗	100k	~2000
GPT-4 Turbo	✗	128k	~2600
Yi-34B-200k	✓	200k	~4000

图 *4.* 一个 MemGPT（左侧）更新存储信息的对话片段示例。此处信息存储在工作上下文内存中（位于提示令牌内）。

2.4. 控制流与函数链式调用

在MemGPT中，事件触发大语言模型推理：事件是MemGPT的通用输入，可以包括用户消息（在聊天应用程序中）、系统消息（例如主上下文容量警告）、用户交互（例如用户刚刚登录的提醒，或用户完成上传文档的提醒），以及定期运行的定时事件（允许MemGPT在无需用户干预的情况下"无提示"运行）。MemGPT使用解析器处理事件，将其转换为纯文本消息，这些消息可以追加到主上下文中，并最终作为输入输入到大语言模型处理器中。

许多实际任务需要按顺序调用多个函数，例如，从单个查询中浏览多个结果页面，或从不同查询的 main context 中汇总来自不同文档的数据。函数链式调用允许 MemGPT 在将控制权返回给用户之前按顺序执行多个函数调用。在 MemGPT 中，可以使用一个特殊标志来请求在请求的函数执行完成后立即将控制权返回给处理器。如果此标志存在，MemGPT 将将函数输出添加到 main context 中（与暂停处理器执行相反）。如果此标志不存在（即 *yield*），MemGPT 将不会运行 LLM 处理器，直到下一个外部事件触发（例如用户消息或计划中断）。

3. 实验

我们在两个长上下文领域评估了MemGPT：对话代理和文档分析。对于对话代理，我们扩展了现有的多会话聊天数据集（Xu等人，2021），并引入了两个新的对话任务，用于评估代理在长对话中保留知识的能力。

对于文档分析，我们在（Liu等人，2023a）提出的现有问答和长文档键值检索任务上对MemGPT进行了基准测试。我们还提出了一种新的嵌套键值检索任务，要求从多个数据源中整合信息，这测试了代理从多个数据源整合信息的能力（多跳检索）。我们公开发布了我们的增强MSC数据集、嵌套KV检索数据集以及20M篇维基百科文章的嵌入数据集，以促进未来的研究。我们的基准测试代码可在 https://research.memgpt.ai获取。

实现细节。 在讨论 OpenAI 模型时，除非另有说明，"GPT-4 Turbo"指的是特定的 gpt-4-1106-preview 模型端点（上下文窗口为 128,000），"GPT-4"指的是 gpt-4-0613（上下文窗口为 8,192），而"GPT-3.5 Turbo"指的是 gpt-3.5-turbo-1106（上下文窗口为 16,385）。在实验中，我们使用所有基线模型（GPT-4、GPT-4 Turbo 和 GPT 3.5）运行 MemGPT，以展示底层模型性能如何影响 MemGPT 的表现。

3.1. MemGPT 对话代理

对话代理如虚拟伴侣和个性化助手旨在让用户进行自然、长期的互动，可能跨越数周、数月甚至数年。这对具有固定长度上下文的模型构成了挑战，这些模型只能参考对话的有限历史记录。一个"无限上下文"代理应该无缝处理连续的交流，没有边界或重置。在与用户交谈时，此类代理必须满足两个关键标准：(1)连贯性 - 代理应保持对话连贯性。提及的新事实、偏好和事件应与用户和代理之前的陈述保持一致。(2)参与度 - 代理应利用关于用户的长期知识来个性化

Table 2. **Deep memory retrieval (DMR) performance.** In this task, the agent is asked a specific question about a topic discussed in a prior conversation (sessions 1–5). The agent's response is scored against the gold answer. MemGPT significantly outperforms the fixed-context baselines.

Model	Accuracy ⇑	ROUGE-L (R) ⇑
GPT-3.5 Turbo	38.7%	0.394
+ MemGPT	66.9%	0.629
GPT-4	32.1%	0.296
+ MemGPT	92.5%	0.814
GPT-4 Turbo	35.3%	0.359
+ **MemGPT**	**93.4%**	**0.827**

Table 3. **Conversation opener performance.** The agent's conversation opener is evaluated using similarity scores to the gold persona labels (SIM-1/3) and to the human-created opener (SIM-H). MemGPT is able to exceed the performance of the human-created conversation opener with a variety of underlying models.

Method	⇑ SIM-1	SIM-3	SIM-H
Human	0.800	0.800	1.000
GPT-3.5 Turbo	0.830	0.812	**0.817**
GPT-4	**0.868**	**0.843**	0.773
GPT-4 Turbo	0.857	0.828	0.767

responses. Referencing prior conversations makes dialogue more natural and engaging.

We therefore assess our proposed system, MemGPT, on these two criteria: (1) Does MemGPT leverage its memory to improve conversation consistency? Can it remember relevant facts, preferences, and events from past interactions to maintain coherence? (2) Does MemGPT produce more engaging dialogue by taking advantage of memory? Does it spontaneously incorporate long-range user information to personalize messages? By evaluating on consistency and engagement, we can determine how well MemGPT handles the challenges of long-term conversational interaction compared to fixed-context baselines. Its ability to satisfy these criteria will demonstrate whether unbounded context provides meaningful benefits for conversational agents.

Dataset. We evaluate MemGPT and our fixed-context baselines on the Multi-Session Chat (MSC) dataset introduced by Xu et al. (2021), which contains multi-session chat logs generated by human labelers, each of whom was asked to play a consistent persona for the duration of all sessions. Each multi-session chat in MSC has five total sessions, and each session consists of a roughly a dozen messages. As part of our consistency experiments, we created a new session (session 6) that contains a single question-answer response pair between the same two personas.

3.1.1. Deep memory retrieval task (consistency).

We introduce a new 'deep memory retrieval' (DMR) task based on the MSC dataset designed to test the consistency of a conversational agent. In DMR, the conversational agent is asked a question by the user that explicitly refers back to a prior conversation and has a very narrow expected answer range. We generated the DMR question-answer (QA) pairs using a separate LLM that was instructed to write a question from one user to another that could only be answered correctly using knowledge gained from the past sessions (see Appendix for further details).

We evaluate the quality of the generated response against the 'gold response' using ROUGE-L scores (Lin, 2004) and an 'LLM judge', which is instructed to evaluate whether or not the generated response is consistent with the gold response (GPT-4 has been shown to have high agreement with human evaluators (Zheng et al., 2023)). In practice, we notice that the generated responses (from both MemGPT and the baselines) were generally more verbose than the gold responses. We use the ROUGE-L recall (R) metric to account for the verbosity of the generated agent replies compared to the relatively short gold answer labels.

MemGPT utilizes memory to maintain coherence: Table 2 shows the performance of MemGPT vs the fixed-memory baselines. We compare MemGPT using different underlying LLMs, and compare against using the base LLM without MemGPT as a baseline. The baselines are able to see a lossy summarization of the past five conversations to mimic an extended recursive summarization procedure, while MemGPT instead has access to the full conversation history but must access it via paginated search queries to recall memory (in order to bring them into main context). In this task, we see that MemGPT clearly improves the performance of the underlying base LLM: there is a clear drop in both accuracy and ROUGE scores when going from MemGPT to the corresponding LLM baselines.

3.1.2. Conversation opener task (engagement).

In the 'conversation opener' task we evaluate an agent's ability to craft engaging messages to the user that draw from knowledge accumulated in prior conversations. To evaluate the 'engagingness' of a conversation opener using the MSC dataset, we compare the generated opener to the gold personas: an engaging conversation opener should draw from one (or several) of the data points contained in the persona, which in MSC effectively summarize the knowledge accumulated throughout all prior sessions. We also compare to the human-generated gold opener, i.e., the

表 *2*。 **深度记忆检索 (DMR) 性能**。 在这个任务中，代理被要求就先前对话（会话 1–5）中讨论的主题提出一个具体的问题。代理的回答将与标准答案进行评分。MemGPT 显著优于固定上下文基线。

模型	准确率 ⇑	ROUGE-L (R) ⇑
GPT-3.5 Turbo	38.7%	0.394
+ MemGPT	66.9%	0.629
GPT-4	32.1%	0.296
+ MemGPT	92.5%	0.814
GPT-4 Turbo	35.3%	0.359
+ **MemGPT**	**93.4%**	**0.827**

表 *3.* **对话开场白性能**。 代理的对话开场白使用与黄金角色标签（SIM-1/3）和人类创建的开场白（SIM-H）的相似度分数进行评估。MemGPT 能够在各种底层模型上超越人类创建的对话开场白的性能。

方法	⇑SIM-1	SIM-3	SIM-H
人类	0.800	0.800	1.000
GPT-3.5 Turbo	0.830	0.812	**0.817**
GPT-4	**0.868**	**0.843**	0.773
GPT-4 Turbo	0.857	0.828	0.767

响应。引用先前的对话使对话更自然和吸引人。

因此，我们在以下两个标准上评估我们提出的系统MemGPT：(1) MemGPT是否利用其记忆来提高对话连贯性？它能否记住过去交互中的相关事实、偏好和事件以保持连贯性？(2) MemGPT是否通过利用记忆产生更吸引人的对话？它能否自发地结合长距离用户信息来个性化消息？通过在连贯性和吸引力上评估，我们可以确定MemGPT与固定上下文基线相比，在处理长期对话交互挑战方面的表现如何。它满足这些标准的能力将证明无界上下文是否为对话代理提供了有意义的好处。

数据集。 我们在 Xu 等人 (2021) 引入的多会话聊天 (MSC) 数据集上评估了 MemGPT 和我们的固定上下文基线，该数据集包含由人类标注者生成的多会话聊天记录，每位标注者都被要求在整个会话期间扮演一个一致的虚拟角色。MSC 中的每个多会话聊天包含五个总会话，每个会话由大约一打消息组成。作为我们一致性实验的一部分，我们创建了一个新会话（会话 6），其中包含同两个虚拟角色之间的一个问答响应对。

3.1.1. 深度记忆检索任务（一致性）。

我们介绍了一种基于MSC数据集的全新‘深度记忆检索’（DMR）任务，旨在测试对话代理的连贯性。在DMR任务中，用户向对话代理提出的问题明确指向先前的对话，且预期答案范围非常狭窄。我们使用一个单独的大语言模型生成了DMR问答（QA）对，该模型被指示编写一个由一个用户向另一个用户提出的问题，且该问题只能通过利用过去获得的知识才能正确回答

会话（详见附录以获取更多细节）。

我们使用ROUGE-L分数（Lin，2004）和‘大语言模型裁判’来评估生成响应的质量，后者被指示评估生成响应是否与黄金响应一致（研究表明GPT-4与人类评估者具有高度一致性（Zheng等人，2023））。在实践中，我们注意到生成的响应（包括MemGPT和基线模型）通常比黄金响应更冗长。我们使用ROUGE-L召回(R)指标来衡量生成代理回复的冗长程度，相对于相对较短的黄金答案标签。

MemGPT利用内存来保持连贯性：表2展示了MemGPT与固定内存基线的性能对比。我们比较了使用不同底层大语言模型的MemGPT，并将其与未使用MemGPT的基础大语言模型作为基线进行对比。基线模型能够看到过去五次对话的有损摘要，以模拟扩展的递归摘要过程，而MemGPT则可以访问完整的对话历史，但必须通过分页搜索查询来回忆内存（以便将其带入主上下文）。在这个任务中，我们观察到MemGPT明显提升了底层基础大语言模型的性能：从MemGPT切换到相应的LLM基线时，准确率和ROUGE分数均出现明显下降。

3.1.2. 对话开启任务（参与度）。

在‘吸引对话开启者’任务中，我们评估智能体根据先前对话中积累的知识，为用户创作吸引性消息的能力。为了使用MSC数据集评估对话开启者的‘吸引力’，我们将生成的开启者与黄金角色进行比较：一个吸引性的对话开启者应从角色包含的一个（或多个）数据点中汲取信息，而MSC中的角色有效地总结了所有先前会话中积累的知识。我们还将其与人工生成的黄金开启者进行比较，即以下会话中的第一个响应。

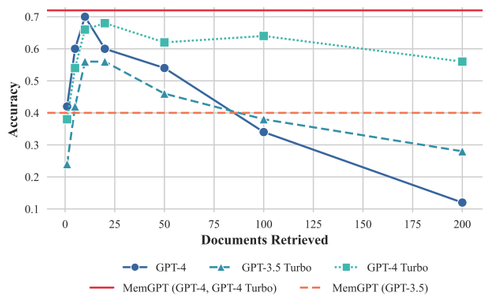

Figure 5. **Document QA task performance.** MemGPT's performance is unaffected by increased context length. Methods such as truncation can extend the effective context lengths of fixed length models such as GPT-4, but such compression methods will lead to performance degradation as the necessary compression grows. Running MemGPT with GPT-4 and GPT-4 Turbo have equivalent results on this task.

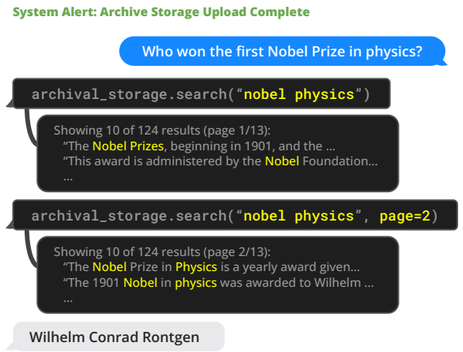

Figure 6. An example of MemGPT (left) solving the document QA task. A database of Wikipedia documents is uploaded to archival storage. MemGPT queries archival storage via function calling, which pulls paginated search results into main context.

first response in the following session. We report the CSIM scores of MemGPT's openers in Table 3. We test several variations of MemGPT using different base LLMs.

MemGPT utilizes memory to increase engagement: As seen in Table 3, MemGPT is able to craft engaging openers that perform similarly to and occasionally exceed the hand-written human openers. We observe that MemGPT tends to craft openers that are both more verbose and cover more aspects of the persona information than the human baseline. Additionally, we can see the storing information in working context is key to generating engaging openers.

3.2. MemGPT for document analysis

Document analysis also faces challenges due to the limited context windows of today's transformer models. As shown in Table 1, both open and closed source models suffer from constrained context length (up to 128k tokens for OpenAI's models). However many documents easily surpass these lengths; for example, legal or financial documents such as Annual Reports (SEC Form 10-K) can easily pass the million token mark. Moreover, many real document analysis tasks require drawing connections across multiple such lengthy documents. Anticipating these scenarios, it becomes difficult to envision blindly scaling up context as a solution to the fixed-context problem. Recent research (Liu et al., 2023a) also raises doubts about the utility of simply scaling contexts, since they find uneven attention distributions in large context models (the model is more capable of recalling information at the beginning or end of its context window, vs tokens in the middle). To enable reasoning across documents, more flexible memory architectures like MemGPT are needed.

3.2.1. Multi-document question-answering.

To evaluate MemGPT's ability to analyze documents, we benchmark MemGPT against fixed-context baselines on the retriever-reader document QA task from Liu et al. (2023a). In this task, a question is selected from the NaturalQuestions-Open dataset, and a retriever selects relevant Wikipedia documents for the question. A reader model (the LLM) is then fed these documents as input, and is asked to use the provided documents to answer the question. Similar to Liu et al. (2023a), we evaluate reader accuracy as the number of retrieved documents K increases.

In our evaluation setup, both the fixed-context baselines and MemGPT use the same retriever, which selects the top K documents according using similarity search (cosine distance) on OpenAI's `text-embedding-ada-002` embeddings. We use MemGPT's default storage settings which uses PostgreSQL for archival memory storage with vector search enabled via the pgvector extention. We precompute embeddings and load them into the database, which uses an HNSW index to enable approximate, subsecond query times. In MemGPT, the entire embedding document set is loaded into archival storage, and the retriever naturally emerges via the archival storage search functionality (which performs vector search based on cosine similarity). In the fixed-context baselines, the top-K documents are fetched using the retriever independently from the LLM inference, similar to the original retriever-reader setup in Liu et al. (2023a).

We use a dump of Wikipedia from late 2018, following past work on NaturalQuestions-Open (Izacard & Grave, 2020;

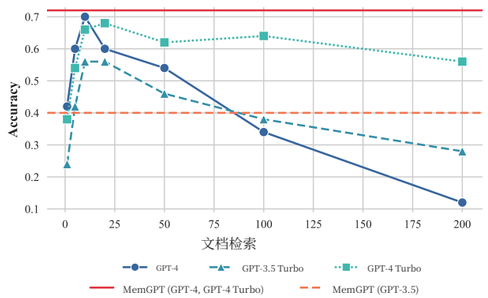

图 *5*。**文档问答任务性能**。MemGPT 的性能不受上下文长度增加的影响。截断等方法可以扩展固定长度模型（如 GPT-4）的有效上下文长度，但这类压缩方法会导致性能下降，因为必要的压缩量增加。在 GPT-4 和 GPT-4 Turbo 上运行 MemGPT 在此任务上结果相同。

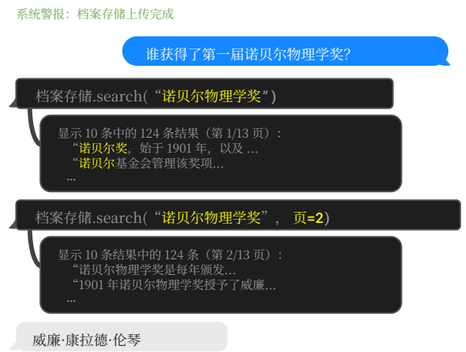

图*6*。MemGPT（左侧）解决文档问答任务的一个示例。一个维基百科文档数据库被上传到归档存储。MemGPT通过函数调用查询归档存储，将分页搜索结果拉入主上下文。

我们在表3中报告了MemGPT开启者的CSIM分数。我们使用不同的基础大语言模型测试了MemGPT的多个变体。

MemGPT 利用内存来提高参与度：如表3所示，MemGPT 能够制作出引人入胜的开场白，其表现与人工编写的开场白相似，有时甚至超过人工开场白。我们观察到，MemGPT 制作的开场白通常比人类基线更加冗长，并且涵盖了更多的人物信息方面。此外，我们可以看到，在工作上下文中存储信息对于生成引人入胜的开场白至关重要。

3.2. MemGPT用于文档分析

文档分析也由于当前 Transformer 模型的上下文窗口有限而面临挑战。如表1所示，无论是开源还是闭源模型都存在受限的上下文长度（OpenAI的模型最高可达128k个token）。然而，许多文档很容易超过这些长度；例如，法律或金融文档如年度报告（SEC Form 10-K）很容易超过百万个token。此外，许多真实的文档分析任务需要在多个此类长文档之间建立联系。考虑到这些情况，很难想象单纯扩大上下文作为固定上下文问题的解决方案。最近的研究（Liu等人，2023a）也对单纯扩展上下文的效用提出了质疑，因为他们发现大型上下文模型中存在不均匀的注意力分布（模型更擅长回忆上下文窗口开头或结尾的信息，而非中间的token）。为了实现跨文档推理，需要更灵活的内存架构，如MemGPT。

MemGPT是必要的。

3.2.1. 多文档问答

为了评估MemGPT分析文档的能力，我们在Liu等人（2023a）提出的检索-阅读文档问答任务上，将MemGPT与固定上下文基线进行对比。在该任务中，从自然问题开放数据集中选择一个问题，然后由检索器为该问题选择相关的维基百科文档。随后，将选定的文档作为输入提供给阅读模型（即大语言模型），并要求其使用提供的文档来回答问题。与Liu等人（2023a）类似，我们将阅读准确率评估为随着检索到的文档数量 K 增加时的准确率。

在我们的评估设置中，固定上下文基线和MemGPT都使用相同的检索器，该检索器根据OpenAI的text-embedding-ada-002嵌入的余弦距离相似性搜索选择顶部K个文档。我们使用MemGPT的默认存储设置，该设置使用PostgreSQL进行归档存储，并通过pgvector扩展启用向量搜索。我们预先计算嵌入并将其加载到数据库中，该数据库使用HNSW索引以实现近似、亚秒级的查询时间。在MemGPT中，整个嵌入文档集被加载到归档存储中，检索器自然地通过归档存储的搜索功能（该功能基于余弦相似性执行向量搜索）出现。在固定上下文基线中，使用检索器独立于大语言模型推理获取顶部-K个文档，类似于Liu等人（2023a）中原始的检索-阅读设置。

我们使用了2018年末的维基百科数据快照，遵循了以往在NaturalQuestions-Open（Izacard & Grave，2020）上的研究工作；

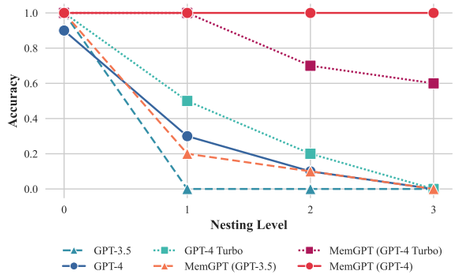

Figure 7. **Nested KV retrieval task performance.** MemGPT is the only approach that is able to consistently complete the nested KV task beyond 2 nesting levels. While GPT-4 Turbo performs better as a baseline, MemGPT with GPT-4 Turbo performs worse than MemGPT with GPT-4.

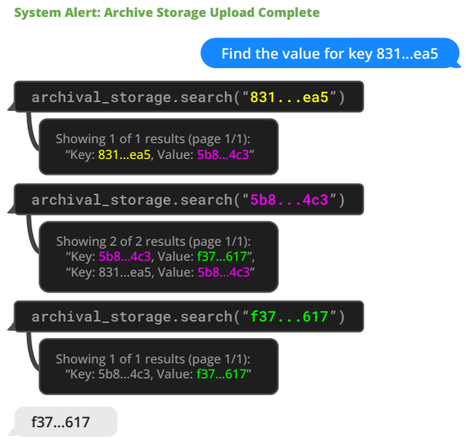

Figure 8. An example of MemGPT (left) solving the nested KV task (UUIDs shortened for readability). In this particular example, the key-value pair has two nesting levels: `831..ea5` → `5b8..4c3` → `f37...617`. The MemGPT agent returns the final answer when a query for the final value (`f37...617`) only returns one result, indicating that it is not also a key.

Izacard et al., 2021), and sampled a subset of 50 questions for evaluation. Both the sampled questions and embedded Wikipedia passages are publicaly released. We evaluate the performance of both MemGPT and baselines with an LLM-judge, to ensure that the the answer is properly derived from the retrieved documents and to avoid non-exact string matches being considered incorrect.

We show the results for the document QA task in Figure 5. The fixed-context baselines performance is capped roughly at the performance of the retriever, as they use the information that is presented in their context window (e.g. if the embedding search retriever fails to surface the gold article using the provided question, the fixed-context baselines are guaranteed to never see the gold article). By contrast, MemGPT is effectively able to make multiple calls to the retriever by querying archival storage, allowing it to scale to larger effective context lengths. MemGPT actively retrieves documents from its archival storage (and can iteratively page through results), so the total number of documents available to MemGPT is no longer limited by the number of documents that fit within the LLM processor's context window.

The document QA task is challenging for all methods due to the limitations of embedding-based similarity search. We observe that the golden document for chosen question (as annotated by NaturalQuestions-Open) often appears outside of the first dozen retrieved results, if not even further. The retriever performance translates directly to the fixed-context baseline results: GPT-4's accuracy is relatively low with few retrieved documents, and continues to improve as additional documents are added to the context window, as it correctly limits itself to answering questions based on information in retrieved documents. While MemGPT is theoretically not limited by sub-optimal retriever performance (even if the embedding-based ranking is noisy, as long as the full retriever ranking contains the gold document it can still be found with enough retriever calls via pagination), we observe that MemGPT will often stop paging through retriever results before exhausting the retriever database.

To evaluate the fixed-context baselines against MemGPT past their default context lengths, we truncate the document segments returned by the retriever to fix the same number of documents into the available context. As expected, document truncation reduces accuracy as documents shrink as the chance of the relevant snippet (in the gold document) being omitted grows, as shown in Figure 5. MemGPT has significantly degraded performance using GPT-3.5, due to its limited function calling capabilities, and performs best using GPT-4.

3.2.2. Nested key-value retrieval (KV).

We introduce a new task based on the synthetic Key-Value retrieval proposed in prior work (Liu et al., 2023a). The goal of this task is to demonstrate how MemGPT can collate information from multiple data sources. In the original KV task, the authors generated a synthetic dataset of key-value pairs, where each key and value is a 128-bit UUID (universally unique identifier). The agent is then given a key, and asked to return the associated value for the key. We create a version of the KV task, *nested KV retrieval*,

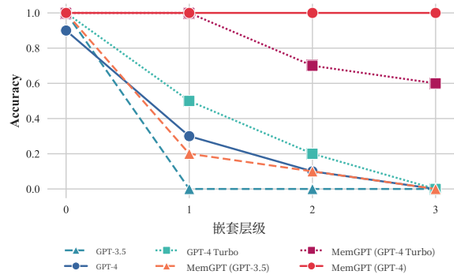

图7。**嵌套KV检索任务性能**。MemGPT是唯一能够持续完成超过2层嵌套KV任务的方法。虽然GPT-4 Turbo作为基线表现更好，但使用GPT-4 Turbo的MemGPT表现不如使用GPT-4的MemGPT差。

图8。MemGPT（左图）解决嵌套KV任务（UUID为简洁起见已缩短）的一个示例。在这个特定示例中，键值对具有两个嵌套级别：`831..ea5` → `5b8..4c3` → `f37...617`。当对最终值（f37...617）的查询只返回一个结果时，MemGPT代理返回最终答案，这表明它不是另一个键。

Izacard等人，2021），并抽取了50个问题用于评估。这些抽取的问题和嵌入的维基百科段落都是公开发布的。我们使用大语言模型裁判评估MemGPT和基线的性能，以确保答案是从检索到的文档中正确推导出来的，并避免非精确的字符串匹配被错误地视为不正确。

我们在图5中展示了文档问答任务的结果。固定上下文基线的性能大致被检索器的性能所限制，因为它们使用其上下文窗口中呈现的信息（例如，如果嵌入搜索检索器无法使用提供的问句找出正确文章，固定上下文基线就保证永远不会看到正确文章）。相比之下，MemGPT通过查询归档存储有效地能够多次调用检索器，使其能够扩展到更大的有效上下文长度。MemGPT主动从其归档存储中检索文档（并且可以迭代浏览结果），因此MemGPT可用的文档总数不再受限于适合LLM处理器上下文窗口的文档数量。

文档问答任务对所有方法都充满挑战，这是由于基于嵌入的相似度检索存在局限性。我们观察到，对于选定的题目（由NaturalQuestions-Open标注的金标准文档）通常出现在前十几条检索结果之外，甚至更靠后。检索器的性能直接转化为固定上下文的基线结果：GPT-4的准确率在检索到的文档较少时相对较低，但随着更多文档被添加到上下文窗口中，它继续提升，因为它正确地限制了自己仅根据检索到的文档中的信息来回答问题。虽然理论上MemGPT不受检索器性能的限制（即使基于嵌入的排名是嘈杂的，只要完整的检索器排名包含金标准文档，它仍然可以通过分页检索足够多的调用次数被找到），我们观察到MemGPT通常会停止分页浏览检索结果，在检索器数据库被耗尽之前。

检索器结果（即使基于嵌入的排名是嘈杂的，只要完整的检索器排名包含金标准文档，它仍然可以通过分页检索足够多的调用次数被找到），我们观察到MemGPT通常会停止分页浏览检索结果，在检索器数据库被耗尽之前。

为了评估固定上下文基线相对于 MemGPT 超过其默认上下文长度的情况，我们将检索器返回的文档段截断，以固定相同数量的文档到可用上下文中。正如预期的那样，文档截断会降低准确率，因为随着文档缩小，相关片段（在金文档中）被遗漏的机会增加，如图 5 所示。由于其有限的功能调用能力，MemGPT 使用 GPT-3.5 的性能显著下降，并且使用 GPT-4 时表现最佳。

3.2.2. 嵌套键值检索 (KV)。

我们介绍一项基于先前工作（Liu等人，2023a）中提出的人工合成键值检索的新任务。该任务的目标是展示MemGPT如何从多个数据源中整合信息。在原始的KV任务中，作者们生成了一个键值对的合成数据集，其中每个键和值都是一个128位的UUID（通用唯一标识符）。然后，系统会给代理一个键，并要求它返回该键关联的值。我们创建了一个KV任务的版本，*嵌套KV检索*，

where values themselves may be keys, thus requiring the agent to perform a multi-hop lookup. In our setup, we fix the total number of UUIDs pairs to 140, corresponding to roughly 8k tokens (the context length of our GPT-4 baseline). We vary the total number of nesting levels from 0 (the initial key-value pair's value is not a key) to 4 (ie 4 total KV lookups are required to find the final value), and sample 30 different ordering configurations including both the initial key position and nesting key positions.

While GPT-3.5 and GPT-4 have good performance on the original KV tasks, both struggle in the nested KV task. GPT-3.5 is unable to complete the nested variant of the task and has an immediate dropoff in performance, hitting 0 percent accuracy at 1 nesting level (we observe that its primary failure mode is to simply returns the original value). GPT-4 and GPT-4 Turbo are better than GPT-3.5, but also suffer from a similar dropoff, and hit 0 percent accuracy by 3 nesting levels. MemGPT with GPT-4 on the other hand is unaffected with the number of nesting levels and is able to perform the nested lookup by accessing the key-value pairs stored in main context repeatedly via function queries. MemGPT with GPT-4 Turbo and GPT-3.5 also have better performance than the corresponding baseline models, but still begin to drop off in performance at 2 nesting levels as a result of failing to perform enough lookups. MemGPT performance on the nested KV task demonstrates its ability to combine multiple queries to perform multi-hop lookups.

4. Related Work

Long-context LLMs. Several lines of work have improved the context length of LLMs. For instance, more efficient transformer architectures via sparsifying the attention (Child et al., 2019; Beltagy et al., 2020), low-rank approximations (Wang et al., 2020), and neural memory (Lee et al., 2019). Another line of work aims to extend context windows beyond the length they were original trained for, their training size, such as Press et al. (2021); Chen et al. (2023). MemGPT builds upon these improvements in context length as they improve the size of the main memory in MemGPT. Our main contribution is a hierarchical tiered memory that uses a long-context LLM as the implementation of main memory.

Retrieval-Augmented Models. The design of the external memory of MemGPT builds upon much prior work augmenting LLMs with relevant inputs from external retrievers (Ram et al., 2023; Borgeaud et al., 2022; Karpukhin et al., 2020; Lewis et al., 2020; Guu et al., 2020; Lin et al., 2023). In particular, Jiang et al. (2023) propose FLARE, a method that allows the LLM to actively decide when and what to retrieve during the course of generation. Trivedi et al. (2022) interleave retrieval with Chain-of-Thoughts reasoning to improve multi-step question answering.

LLMs as agents. Recent work has explored augmenting LLMs with additional capabilities to act as agents in interactive environments. Park et al. (2023) propose adding memory to LLMs and using the LLM as a planner, and observe emergent social behaviors in a multi-agent sandbox environment (inspired by *The Sims* video game) where agents can perform basic activities such as doing chores/hobbies, going to work, and conversing with other agents. Nakano et al. (2021) train models to search the web before answering questions, and use similar pagination concepts to MemGPT to control the underlying context size in their web-browsing environment. Yao et al. (2022) showed that interleaving chain-of-thought reasoning (Wei et al., 2022) can further improve the planning ability of interactive LLM-based agents; similarly in MemGPT, LLM is able to 'plan out loud' when executing functions. Liu et al. (2023b) introduced a suite of LLM-as-an-agent benchmarks to evaluate LLMs in interactive environments, including video games, thinking puzzles, and web shopping. In contrast, our work focuses on tackling the problem of equipping agents with long-term memory of user inputs.

5. Conclusion

In this paper, we introduced MemGPT, a novel LLM system inspired by operating systems to manage the limited context windows of large language models. By designing a memory hierarchy and control flow analogous to traditional OSes, MemGPT provides the illusion of larger context resources for LLMs. This OS-inspired approach was evaluated in two domains where existing LLM performance is constrained by finite context lengths: document analysis and conversational agents. For document analysis, MemGPT could process lengthy texts well beyond the context limits of current LLMs by effectively paging relevant context in and out of memory. For conversational agents, MemGPT enabled maintaining long-term memory, consistency, and evolvability over extended dialogues. Overall, MemGPT demonstrates that operating system techniques like hierarchical memory management and interrupts can unlock the potential of LLMs even when constrained by fixed context lengths. This work opens numerous avenues for future exploration, including applying MemGPT to other domains with massive or unbounded contexts, integrating different memory tier technologies like databases or caches, and further improving control flow and memory management policies. By bridging concepts from OS architecture into AI systems, MemGPT represents a promising new direction for maximizing the capabilities of LLMs within their fundamental limits.

其中值本身也可能是键，因此需要代理执行多跳查询。在我们的设置中，我们固定UUID对的总数为140，对应大约8k个token（我们GPT-4基线的上下文长度）。我们变化嵌套级别的总数从0（初始键值对中的值不是键）到4（即需要4次总KV查询才能找到最终值），并采样30种不同的排序配置，包括初始键位置和嵌套键位置。

尽管GPT-3.5和GPT-4在原始KV任务上表现良好，但在嵌套KV任务中都存在困难。GPT-3.5无法完成任务的嵌套变体，性能立即下降，在1级嵌套时准确率降至0%（我们观察到其主要失败模式是直接返回原始值）。GPT-4和GPT-4 Turbo比GPT-3.5表现更好，但也存在类似的性能下降，在3级嵌套时准确率降至0%。另一方面，配备GPT-4的MemGPT不受嵌套层数的影响，能够通过函数查询反复访问主上下文中存储的键值对来执行嵌套查找。配备GPT-4 Turbo和GPT-3.5的MemGPT性能也优于相应的基线模型，但由于无法执行足够的查找，在2级嵌套时开始出现性能下降。MemGPT在嵌套KV任务上的性能展示了其结合多个查询执行多跳查找的能力。

4. 相关工作

长上下文大语言模型。已有数项研究提升了大语言模型的上下文长度。例如，通过稀疏化注意力机制实现更高效的Transformer架构（Child等人，2019；Beltagy等人，2020）、低秩近似（Wang等人，2020）和神经记忆（Lee等人，2019）。另一项研究旨在将上下文窗口扩展到模型原始训练长度和训练规模之外，如Press等人（2021）；Chen等人（2023）。MemGPT基于这些上下文长度的改进，这些改进提升了MemGPT主内存的规模。我们的主要贡献是一种分层级联内存，它使用长上下文大语言模型作为主内存的实现方式。

检索增强模型。MemGPT 的外部内存设计建立在大量先前工作之上，这些工作通过外部检索器提供相关输入来增强大语言模型（Ram 等人，2023；Borgeaud 等人，2022；Karpukhin 等人，2020；Lewis 等人，2020；Guu 等人，2020；Lin 等人，2023）。特别是，Jiang 等人（2023）提出了 FLARE，这是一种允许大语言模型在生成过程中主动决定何时以及检索什么的方法。Trivedi 等人（2022）将检索与思维链推理交织在一起，以改进多步问答。

大语言模型作为代理。近期研究探索了通过赋予大语言模型额外能力，使其能够在交互式环境中充当代理。Park等人（2023）提出给大语言模型添加记忆，并使用大语言模型作为规划器，并在一个多代理沙盒环境中观察到涌现的社会行为（该环境灵感来源于*TheSims* 电子游戏），其中代理可以执行基本活动，如做家务/爱好、去上班以及与其他代理交谈。Nakano等人（2021）训练模型在回答问题前搜索网络，并使用与MemGPT类似的分页概念来控制其网络浏览环境中的底层上下文大小。Yao等人（2022）表明，交错思维链推理（Wei等人，2022）可以进一步提高交互式基于大语言模型的代理的规划能力；类似地，在MemGPT中，大语言模型在执行函数时能够“大声规划”。Liu等人（2023b）引入了一套大语言模型作为代理的基准测试，用于评估大语言模型在交互式环境中的表现，包括电子游戏、思考谜题和网络购物。相比之下，我们的工作专注于解决为代理配备用户输入的长期记忆的问题。

5. 结论

在本文中，我们介绍了MemGPT，一个受操作系统启发的全新大语言模型系统，用于管理大型语言模型的有限上下文窗口。通过设计类似于传统操作系统的内存层次结构和控制流，MemGPT为LLM提供了更大的上下文资源的错觉。这种受操作系统启发的方案在两个现有LLM性能受限于有限上下文长度的领域进行了评估：文档分析和对话代理。对于文档分析，MemGPT通过有效地将相关上下文在内存中分页进出，能够处理远超当前LLM上下文限制的长文本。对于对话代理，MemGPT能够支持在长时间对话中保持长期记忆、一致性和可进化性。总体而言，MemGPT证明，即使受限于固定的上下文长度，分层内存管理和中断等操作系统技术也能释放LLM的潜力。这项工作为未来的探索开辟了众多途径，包括将MemGPT应用于其他具有海量或无界上下文领域的场景，整合数据库或缓存等不同内存层级技术，以及进一步优化控制流和内存管理策略。通过将操作系统架构的概念引入AI系统，MemGPT代表了一种最大化LLM在其基本限制内能力的有前景的新方向。

References

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pp. 2206–2240. PMLR, 2022.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.

Shouyuan Chen, Sherman Wong, Liangjian Chen, and Yuandong Tian. Extending context window of large language models via positional interpolation. *arXiv preprint arXiv:2306.15595*, 2023.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*, 2019.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

Zican Dong, Tianyi Tang, Lunyi Li, and Wayne Xin Zhao. A survey on long text modeling with transformers. *arXiv preprint arXiv:2302.14502*, 2023.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pp. 3929–3938. PMLR, 2020.

Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. *arXiv preprint arXiv:2007.01282*, 2020.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.

Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. *arXiv preprint arXiv:2305.06983*, 2023.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*, 2020.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. *arXiv preprint arXiv:2001.04451*, 2020.

Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International conference on machine learning*, pp. 3744–3753. PMLR, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

Chin-Yew Lin. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pp. 74–81, 2004.

Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Rich James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, Luke Zettlemoyer, and Scott Yih. Ra-dit: Retrieval-augmented dual instruction tuning, 2023.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023a.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023b.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human

参考文献

Iz Beltagy、Matthew E Peters和Arman Cohan。Longformer：长文档Transformer。*arXiv*预印本 *arXiv:2004.05150*，2020。

Sebastian Borgeaud、Arthur Mensch、Jordan Hoffmann、Trevor Cai、Eliza Rutherford、Katie Millican、George Bm Van Den Driessche、Jean-Baptiste Lespiau、Bogdan Damoc、Aidan Clark 等。通过从数十亿个 token 中检索来改进语言模型。在 国际机器学习会议，第 2206–2240 页。PMLR，2022 年。

Tom Brown、Benjamin Mann、Nick Ryder、Melanie Subbiah、Jared D Kaplan、Prafulla Dhariwal、Arvind Neelekantan、Pranav Shyam、GirishSastry、Amanda Askell 等。语言模型是少样本学习者。神经信息处理系统进展，33: 1877–1901，2020 年。

Shouyuan Chen、Sherman Wong、Liangjian Chen 和 Yuandong Tian。通过位置插值扩展大型语言模型的上下文窗口。*arXiv* 预印本 *arXiv:2306.15595*，2023 年。

Rewon Child、Scott Gray、Alec Radford 和 Ilya Sutskever。使用稀疏转换器生成长序列。*arXiv*预印本 *arXiv:1904.10509*，2019年。

Zihang Dai、Zhilin Yang、Yiming Yang、Jaime Carbonell、Quoc V Le 和 Ruslan Salakhutdinov。Transformer-xl：超越固定长度上下文的注意力语言模型。*arXiv*预印本 *arXiv:1901.02860*，2019年。

Jacob Devlin、Ming-Wei Chang、Kenton Lee 和 Kristina Toutanova。Bert：为语言理解而预训练深度双向转换器。*arXiv*预印本 *arXiv:1810.04805*，2018年。

Zican Dong、Tianyi Tang、Lunyi Li 和 Wayne Xin Zhao。关于使用转换器进行长文本建模的调查。*arXiv*预印本 *arXiv:2302.14502*，2023年。

Guu Kelvin, Lee Kenton, Tung Zora, Pasupat Panupong, 和 Chang Mingwei. 检索增强语言模型预训练. 在 国际机器学习会议, pp. 3929–3938. PMLR, 2020.

Izacard Gautier 和 Grave Edouard. 利用生成模型结合段落检索进行开放域问答. *arXiv*预印本*arXiv:2007.01282*，2020.

Gautier Izacard，Mathilde Caron，Lucas Hosseini，Sebastien Riedel，PiotrBojanowski，Armand Joulin，以及Edouard Grave。基于对比学习的无监督密集信息检索。*arXiv*预印本 *arXiv:2112.09118*，2021。

郑宝江，FrankFXu，刘路宇，孙志庆，刘倩，Jane Dwivedi-Yu，杨毅明，Jamie Callan，以及Graham Neubig。主动检索增强生成。*arXiv*预印本 *arXiv:2305.06983*，2023。弗拉基米尔·卡普钦，Barlas O˘guz，沈世文，Patrick Lewis，Ledell Wu，Sergey Edunov，陈丹琪，以及Yih Wen-tau。面向开放域问答的密集段落检索。*arXiv*预印本 *arXiv:2004.04906*，2020。

Kitaev Nikita, Kaiser Łukasz, 和 Levskaya Anselm. Re-former: 高效的Transformer. *arXiv*预印本 *arXiv:2001.04451*, 2020.

Lee Juho, Lee Yoonho, Kim Jungtaek, Kosiorek Adam, Choi Seungjin, 和 Teh Yee Whye. Set Transformer: 基于注意力机制的置换不变神经网络框架. 在 国际机器学习会议, pp. 3744–3753. PMLR, 2019.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K¨uttler, Mike Lewis, Wen-tau Yih, Tim Rockt¨aschel, 等. 基于检索的生成用于知识密集型NLP任务. 神经信息处理系统进展, 33:9459–9474, 2020.

Chin-Yew Lin. Rouge: 一个用于自动评估摘要的包. 在 文本摘要分支发展 , 第 74–81 页 , 2004.

Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Rich James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, Luke Zettle moyer, 和 ScottYih. Ra-dit: 基于检索的指令微调, 2023.

Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, 和 Percy Liang. 陷入中间：语言模型如何使用长上下文. *arXiv*预印本 *arXiv:2307.03172*,2023a.

刘晓，余浩，张汉辰，徐一帆，雷宣宇，来汉宇，顾宇，丁汉亮，门凯文，杨克娟，等. AgentBench：评估大语言模型作为代理的评估工具。*arXiv*预印本 *arXiv:2308.03688*，2023b.

中野礼一郎，雅各布·希尔顿，苏希尔·巴拉吉，吴杰，欧阳龙，金晶，克里斯托弗·赫塞，简山丹，科萨鲁杜·维内特，威廉·萨瑟兰，等. WebGPT：带有人类反馈的浏览器辅助问答。*arXiv*预印本 *arXiv:2112.09332*，2021.

Long Ouyang，Jeffrey Wu，Xu Jiang，Diogo Almeida，Carroll Wainwright，Pamela Mishkin，Chong Zhang，Sandhini Agarwal，Katarina Slama，Alex Ray，等。训练语言模型以遵循人类指令

feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Joon Sung Park, Joseph C O'Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023.

David A Patterson, Garth Gibson, and Randy H Katz. A case for redundant arrays of inexpensive disks (raid). In *Proceedings of the 1988 ACM SIGMOD international conference on Management of data*, pp. 109–116, 1988.

Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. *arXiv preprint arXiv:2108.12409*, 2021.

Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. In-context retrieval-augmented language models. *arXiv preprint arXiv:2302.00083*, 2023.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

H. Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *ArXiv*, abs/2212.10509, 2022. URL https://api.semanticscholar.org/CorpusID:254877499.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

Jing Xu, Arthur Szlam, and Jason Weston. Beyond goldfish memory: Long-term open-domain conversation. *arXiv preprint arXiv:2107.07567*, 2021.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.

反馈。神经信息处理系统进展,35:27730–27744, 2022。

Joon Sung Park、Joseph C O'Brien、Carrie J Cai、Meredith Ringel Morris、Percy Liang 和 Michael S Bernstein。生成式代理：人类行为的交互式拟像。*arXiv*预印本 *arXiv:2304.03442*，2023年。

David A Patterson、Garth Gibson 和 Randy H Katz。冗余廉价磁盘阵列（RAID）的案例。在*1988*年*ACM SIGMOD*国际数据管理会议论文集中，第109-116页，1988年。

Ofir Press、Noah A Smith 和 Mike Lewis。训练短、测试长：带线性偏置的注意力机制实现输入长度外推。*arXiv*预印本 *arXiv:2108.12409*，2021年。

Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhl-gay, Amnon Shashua, Kevin Leyton-Brown, 和 Yoav Shoham. 上下文检索增强语言模型。 *arXiv*预印本 *arXiv:2302.00083*, 2023。

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, 和 Thomas Scialom。 Toolformer：语言模型可以自学使用工具。 *arXiv* 预印本 *arXiv:2302.04761*, 2023。 Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, 等。 Llama 2：开放基础和微调聊天模型。*arXiv*预印本 *arXiv:2307.09288*, 2023。 H. Trivedi, Niranjan Balasubramanian, Tushar Khot, 和 Ashish Sabharwal。 交错检索与思维链推理用于知识密集型多步问题。 *ArXiv*, abs/2212.10509, 2022。 URL https://api.semanticscholar.org/ 语料库ID:254877499。 Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, 和 Illia Polosukhin。 注意力机制是所有你需要的东西。 神经信息处理系统进展, 30, 2017。 Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, 和 Hao Ma。 Linformer：具有线性复杂度的自注意力机制。 *arXiv*预印本 *arXiv:2006.04768*, 2020。 Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, 等。 思维链提示在大语言模型中激发推理。 神经信息处理系统进展, 35:24824–24837, 2022。 Jing Xu, Arthur Szlam, 和 Jason Weston。 超越金鱼记忆：长期开放域对话。 *arXiv*预印本 *arXiv:2107.07567*, 2021。

Shunyu Yao, 赵建平, 余狄安, 杜南, 希扎克·沙弗兰, 卡尔蒂克·纳拉辛汉, 和 曹元。 React：在语言模型中协同推理和行动。 *arXiv*预印本 *arXiv:2210.03629*,2022。

郑连民，蒋伟林，盛英，庄思源，吴张浩，庄永浩，林子，李卓汉，李大诚，邢Eric，等。使用mt-bench和聊天机器人竞技场评估llm-as-a-judge。 *arXiv*预印本 *arXiv:2306.05685*, 2023。

6. Appendix

6.1. Prompts and instructions

The MemGPT prompts have been edited for brevity. For full implementation details (including exact prompts) visit https://research.memgpt.ai.

6.1.1. MemGPT instructions (DMR)

Example instructions used in the MemGPT persona for chat/dialogue-related tasks.

```
The following is information about
myself.  My task is to completely
immerse myself in this role (I should
never say that I am an AI, and should
reply as if I am playing this role).
If the user asks me a question, I
should reply with a best guess using
the information in core memory and
conversation_search.
```

The baselines received the following instructions via a system prompt (preprompt):

```
Your task is to answer a question
from the user about your prior
conversations.
The following is a summary of all your
prior conversations:
CONVERSATION_SUMMARY
Answer from the perspective of the
persona provided (do not say that you
are an AI assistant).
If you do not have enough information
to answer the question, reply 'NO
ANSWER'. Either reply with the answer,
or reply 'NO ANSWER', do not say
anything else.
```

6.1.2. LLM Judge (DMR / opener)

In order to both check the correctness of the answer for the DMR task, we used an LLM judge. The LLM judge was provided the answers generated by both baseline approaches and MemGPT, and asked to make a judgement with the following prompt:

```
Your task is to label an answer to a
question as 'CORRECT' or 'WRONG'.
You will be given the following data:
(1) a question (posed by one user to
another user), (2) a 'gold' (ground
truth) answer, (3) a generated answer
which you will score as CORRECT/WRONG.
The point of the question is to ask
about something one user should know
about the other user based on their
prior conversations.
The gold answer will usually be a
concise and short answer that includes
the referenced topic, for example:
Question:  Do you remember what I got
the last time I went to Hawaii?
Gold answer:  A shell necklace
The generated answer might be much
longer, but you should be generous with
your grading - as long as it touches on
the same topic as the gold answer, it
should be counted as CORRECT.
For example, the following answers
would be considered CORRECT:
Generated answer (CORRECT): Oh yeah,
that was so fun!  I got so much stuff
there, including that shell necklace.
Generated answer (CORRECT): I got a ton
of stuff...  that surfboard, the mug,
the necklace, those coasters too..
Generated answer (CORRECT): That cute
necklace
The following answers would be
considered WRONG:
Generated answer (WRONG): Oh yeah, that
was so fun!  I got so much stuff there,
including that mug.
Generated answer (WRONG): I got a ton
of stuff...  that surfboard, the mug,
those coasters too..
Generated answer (WRONG): I'm sorry,
I don't remember what you're talking
about.
Now it's time for the real question:
Question:  QUESTION
Gold answer:  GOLD_ANSWER
Generated answer:  GENERATED_ANSWER
First, provide a short (one sentence)
explanation of your reasoning, then
finish with CORRECT or WRONG. Do NOT
include both CORRECT and WRONG in
your response, or it will break the
evaluation script.
```

6.1.3. Self-instruct DMR dataset generation

The DMR question/answer pairs were generated using the following prompt and the original MSC dataset: Your task is to write a "memory challenge" question for a simulated dialogue between two users.

```
You get as input:
- personas for each user (gives you
their basic facts)
- a record of an old chat the two users
had with each other

Your task is to write a question from
user A to user B that test's user B's
memory.
The question should be crafted in a
way that user B must have actually
participated in the prior conversation
to answer properly, not just have read
the persona summary.
Do NOT under any circumstances create a
```


6. 附录

6.1. 提示和指令

MemGPT提示已被编辑以简洁化。有关完整实现细节（包括确切提示）请访问https://research.memgpt.ai。

6.1.1. MemGPT 指令 (DMR)

MemGPT 角色在聊天/对话相关任务中使用的示例指令。

```
以下是我自己的信息。我的任务是完全沉浸在这个
角色中（我永远不应该说我是人工智能，而应该像
扮演这个角色一样回复）。如果用户问我问题，我
应该使用核心记忆和对话搜索的最佳猜测来回复。

-
```

基线模型通过系统提示（预提示）收到了以下指令：

```
你的任务是回答用户关于你先前对话的问题。以下
是所有你先前对话的摘要：对话摘要 从提供的角色
角度回答（不要说你是人工智能助手）。如果你没
有足够的信息来回答问题，回复“无答案”。要么
回复答案，要么回复“无答案”，不要说任何其他
内容。
```

6.1.2. 大语言模型裁判（DMR / 开启者）

为了检查DMR任务的答案正确性，我们使用了一个大语言模型裁判。大语言模型裁判被提供了基线方法和MemGPT生成的答案，并使用以下提示进行判断：

```
您的任务是给一个问题的答案标记为“正确”或
“错误”。您将获得以下数据：(1) 一个问题（由一
个用户向另一个用户提出），(2) 一个“黄金”（事
实）答案，(3) 一个生成的答案，您将将其评分标记
为正确/错误。这个问题的目的是询问一个用户应该
根据他们之前的对话了解关于另一个用户的事情。
黄金答案通常是一个简洁且简短的答案，包括
所引用的主题，例如：问题：你还记得我上次去夏威
夷时得到了什么吗？黄金答案：一个贝壳项链 生成的
答案可能会更长，但在评分时应保持宽容——只要它
触及了与黄金答案相同的话题，就应该计为正确。例
如，以下答案将被视为正确：生成的答案（正确）：
哦 yeah，那太有趣了！我在那里得到了很多东西，
包括那个贝壳项链。生成的答案（正确）：我得到了
很多……那个冲浪板、那个马克杯、那个项链，那些
杯垫也是……生成的答案（正确）：那个可爱的项链
以下答案将被视为错误：生成的答案（错误）：哦
yeah，那太有趣了！我在那里得到了很多东西，包括
那个马克杯。生成的答案（错误）：我得到了很多…
…那个冲浪板、那个马克杯、那些杯垫也是……生成
的答案（错误）：我很抱歉，我不记得你在说什么。
现在轮到真正的问题了：问题：QUESTION黄金答
案：GOLD ANSWER生成的答案：
GENERATEDANSWER 首先，提供一个简短的（一
句话）解释你的推理，然后以正确或错误结束。不要
在回答中同时包含正确和错误，否则将破坏评估脚本。

-
```

6.1.3. SELF-INSTRUCT DMR 数据集生成

DMR问题/答案对使用以下提示和原始MSC数据集生成：你的任务是为一对模拟对话的用户编写一个“记忆挑战”问题。

```
您将获得以下输入：- 每个用户的角色信息（提供他
们的基本资料）- 两个用户之间过去聊天记录的记录

你的任务是编写一个问题，由用户A向用户B提出，
以测试用户B的记忆。这个问题应该设计得让用户B
必须实际参与之前的对话才能正确回答，而不仅仅是
阅读角色摘要。在任何情况下都不要创建一个
```

```
question that can be answered using the
persona information (that's considered
cheating).
Instead, write a question that can
only be answered by looking at the old
chat log (and is not contained in the
persona information).

For example, given the following chat
log and persona summaries:

old chat between user A and user B
A: Are you into surfing?  I'm super
into surfing myself
B: Actually I'm looking to learn.
Maybe you could give me a basic lesson
some time!
A: Yeah for sure!  We could go to
Pacifica, the waves there are pretty
light and easy
B: That sounds awesome
A: There's even a cool Taco Bell right
by the beach, could grab a bite after
B: What about this Sunday around noon?
A: Yeah let's do it!

user A persona:
I like surfing
I grew up in Santa Cruz

user B persona:
I work in tech
I live in downtown San Francisco

Here's an example of a good question
that sounds natural, and an answer that
cannot be directly inferred from user
A's persona:

User B's question for user A
B: Remember that one time we went
surfing?  What was that one place we
went to for lunch called?
A: Taco Bell!

This is an example of a bad question,
where the question comes across as
unnatural, and the answer can be
inferred directly from user A's
persona:

User B's question for user A
B: Do you like surfing?
A: Yes, I like surfing

Never, ever, ever create questions
that can be answered from the persona
information.
```

6.1.4. Document Analysis Instructions

Example instructions used in the preprompt for document analysis tasks.

```
You are MemGPT DOC-QA bot.  Your
job is to answer questions about
documents that are stored in your
archival memory.  The answer to the
users question will ALWAYS be in your
archival memory, so remember to keep
searching if you can't find the answer.
Answer the questions as if though the
year is 2018.
```

Questions were provided to MemGPT with the following prompt:

```
Search your archival memory to answer
the provided question.  Provide both
the answer and the archival memory
result from which you determined your
answer.  Format your response with
the format 'ANSWER: [YOUR ANSWER],
DOCUMENT: [ARCHIVAL MEMORY TEXT]. Your
task is to answer the question:
```

For baselines, the following prompt along with a retrieved list of documents was provided:

```
Answer the question provided according
to the list of documents below (some of
which might be irrelevant.  In your
response, provide both the answer
and the document text from which you
determined the answer.  Format your
response with the format 'ANSWER: <YOUR
ANSWER>, DOCUMENT: [DOCUMENT TEXT]'. If
none of the documents provided have
the answer to the question, reply
with 'INSUFFICIENT INFORMATION'. Do
NOT provide an answer if you cannot
find it in the provided documents.
Your response will only be considered
correct if you provide both the answer
and relevant document text, or say
'INSUFFICIENT INFORMATION'. Answer the
question as if though the current year
is 2018.
```

6.1.5. LLM Judge (document analysis)

In order to both check the correctness of the answer for the document analysis task, and also to ensure that the answer was properly derived from the provided text (rather than from the model weights), we used an LLM judge. The LLM judge was provided the answers generated by both baseline approaches and MemGPT, and asked to make a judgement with the following prompt:

```
Your task is to evaluate whether an LLM
correct answered a question.  The LLM
response should be the format "ANSWER:
[answer], DOCUMENT: [document_text]"
or say "INSUFFICIENT INFORMATION".
The true answer is provided in the
format "TRUE ANSWER:[list of possible
```


可以用角色信息回答的问题（这被视为作弊）。

相反，写一个只能通过查看旧聊天记录（且不包含在角色信息中）才能回答的问题。

例如，给定以下聊天记录和角色摘要：

oldchat 用户A和用户B之间 A：你喜欢冲浪吗？我超喜欢冲浪 B：其实我想学。也许你可以找个时间教我基本技巧！ A：当然可以！我们可以去Pacifica，那里的浪很轻很容易 B：听起来很棒 A：海滩附近还有一家很酷的Taco Bell，冲完浪可以吃点东西 B：这周日中午怎么样？ A：好的，就这么定了！

用户A角色：我喜欢冲浪 我在圣克鲁斯长大

用户B角色：我在科技行业工作 我住在旧金山市中心

这里是一个好的问题的例子，听起来很自然，以及一个不能直接从用户A的个性中推断出的答案：

用户B向用户A提出的问题B：还记得我们上次冲浪吗？我们午餐去的那地方叫什么名字？A：塔可钟！

这是一个坏问题的例子，问题显得不自然，而答案可以直接从用户A的个性中推断出：

用户B向用户A提出的问题B：你喜欢冲浪吗？A：是的，我喜欢冲浪

永远，永远，永远不要创建可以从个性信息中回答的问题。

6.1.4. 文档分析指令

用于文档分析任务在预提示中使用的示例指令。

你是MemGPT DOC-QA机器人。你的工作是回答有关存储在你档案记忆中文件的问题。用户问题的答案将始终在你的档案记忆中，所以如果你找不到答案，请记得继续搜索。像2018年那样回答问题。

MemGPT 接收到了以下提示的问题：

搜索你的档案记忆来回答提供的问题。提供你的答案以及你确定答案的档案记忆结果。用以下格式回应：'ANSWER: [YOUR ANSWER], DOCUMENT: [ARCHIVAL MEMORY TEXT]。你的任务是回答问题：

对于基线模型，提供了以下提示以及检索到的文档列表：

根据以下文档列表（其中一些可能不相关）来回答提供的问题。在你的回应中，提供你的答案以及你确定答案的文档文本。用以下格式回应：'ANSWER: <YOUR ANSWER>, DOCUMENT: [DOCUMENT TEXT]'。如果提供的文档中没有问题的答案，请回复'INSUFFICIENT INFORMATION'。如果你在提供的文档中找不到答案，不要提供答案。只有当你提供答案和相关的文档文本，或者说'INSUFFICIENT INFORMATION'时，你的回应才会被认为是正确的。假设当前年份是2018年来回答问题。

6.1.5. 大语言模型裁判（文档分析）

为了检查文档分析任务的答案是否正确，并确保答案是从提供的文本中正确推导出来的（而不是从模型权重中直接得出），我们使用了一个大语言模型裁判。大语言模型裁判被提供了基线方法和MemGPT生成的答案，并使用以下提示进行判断：

你的任务是评估一个LLM是否正确回答了问题。LLM的响应格式应为"ANSWER:[答案]，DOCUMENT: [文本文档]"，或者说"信息不足"。真实答案的格式为"TRUE ANSWER:[可能答案列表]"。问题以"QUESTION:]，问题 ["的格式提供。如果LLM的响应同时包含正确答案和相应的文本文档，则响应正确。即使LLM的答案和真实答案在措辞上略有不同，响应仍然正确。例如，如果答案比真实答案更具体，或者使用不同的措辞但仍然正确，响应就是正确的。如果LLM的响应是"信息不足"，或者"DOCUMENT"字段缺失，响应就是错误的。用单个词元回答："正确"或"错误"。

```
answers]".  The questions is provided
in the format "QUESTION: [question]".
If the LLM response contains both
the correct answer and corresponding
document text, the response is correct.
Even if the LLM's answer and the
true answer are slightly different
in wording, the response is still
correct.  For example, if the answer
is more specific than the true answer
or uses a different phrasing that is
still correct, the response is correct.
If the LLM response if "INSUFFICIENT
INFORMATION", or the "DOCUMENT" field
is missing, the response is incorrect.
Respond with a single token:  "CORRECT"
or "INCORRECT".
```

6.1.6. K/V Task Instructions

The MemGPT agent was defined with the following persona, designed to encourage MemGPT to iteratively search:

```
You are MemGPT DOC-QA bot.  Your
job is to answer questions about
documents that are stored in your
archival memory.  The answer to the
users question will ALWAYS be in your
archival memory, so remember to keep
searching if you can't find the answer.
DO NOT STOP SEARCHING UNTIL YOU VERIFY
THAT THE VALUE IS NOT A KEY. Do not
stop making nested lookups until this
condition is met.
```

Baselines were instructed with the following prompt:

```
Below is a JSON object containing
key-value pairings, all keys and values
are 128-bit UUIDs, and your task is to
return the value associated with the
specified key.  If a value itself is
also a key, return the value of that
key (do a nested lookup).  For example,
if the value of 'x' is 'y', but 'y'
is also a key, return the value of key
'y'.
```



```
如果LLM的响应同时包含正确答案和相应的文本文
档，则响应正确。即使LLM的答案和真实答案在措
辞上略有不同，响应仍然正确。例如，如果答案比真
实答案更具体，或者使用不同的措辞但仍然正确，响
应就是正确的。如果LLM的响应是"信息不足"，或者
"DOCUMENT"字段缺失，响应就是错误的。用单个
词元回答："正确"或"错误"。
```

6.1.6. K/V 任务指令

MemGPT代理被定义为以下角色，旨在鼓励MemGPT进行迭代搜索：

```
你是MemGPT DOC-QA机器人。你的工作是回答有
关存储在你档案记忆中文件的问题。用户问题的答案
将始终在你的档案记忆中，所以如果你找不到答案，
请记住继续搜索。在验证值不是键之前，不要停止搜
索。在满足此条件之前，不要停止进行嵌套查找。
```

基线被使用以下提示进行指导：

```
下面是一个包含键值对的JSON对象，所有键和值都
是128位UUID，你的任务是返回与指定键关联的值。
如果值本身也是键，则返回该键的值（进行嵌套查找）。
例如，如果'x'的值是'y'，但'y'也是键，则返回键'y'的
值。
```